Measurement time from input–output theory: mathematics and implementation of `updatedLindbladSolver.py`

Companion document to the student notes "Measurement rate from input–output theory"

October 27, 2026

Contents

1 Overview: what the code computes and how the pieces fit 2

2 The quantities to be computed 3

3 Vectorisation: turning the Lindblad equation into a matrix 4

4 Quantum regression on a time grid 5

5 Backend 1: diagonalisation (`method='eig'`) 6

6 Backend 2: adjoint propagation (`method='sparse'`, default) 7

7 Integrating the correlators: the double integral 8

8 Assembling the SNR, the phase, and the measurement time 9

9 Sweeps 10

10 Dense versus sparse matrices, and the other options 11

11 Reference of all options 13

12 Accuracy, limits and validation 14

13 Map from mathematics to code 15

1 Overview: what the code computes and how the pieces fit

The code answers one question: *how long must a homodyne record of the reflected microwave field be integrated before the two states* $|0\rangle, |1\rangle$ *of the system qubit S can be told apart?* The theory notes reduce this to two numbers per initial state of S, a mean and a variance of the integrated quadrature, and from them a signal-to-noise ratio (SNR). The code evaluates these numbers *exactly* for the full model (cavity + detector qubit D + system qubit S), with no dispersive or other approximation on H.

The calculation is a pipeline. Every box below is a function (or a small group of functions) described in this document.

Step	What happens	Functions
1	Build H and collapse operators c_k (your original code)	`Hamiltonian`, `makeCollapseOps`
2	Turn the master equation into a matrix L acting on $\mathrm{vec}(\rho)$	`buildLiouvillian`
3	Compute $\langle a\rangle(t)$ and the two-time tables $\langle a(t'+s)a(t')\rangle$, $\langle a^\dagger(t'+s)a(t')\rangle$ by quantum regression	`_regressionSparse` **or** `_regressionEig` (class `LiouvillianSpectrum`)
4	Subtract means → connected correlators; integrate them over the square $[0,\tau]^2$	`regressionData`, `doubleIntegralCumulative`
5	Assemble SNR(τ), optimise the homodyne phase ϕ, find τ_m	`snrFromData`, `measurementTime`, `calculateSNR`
6	Repeat over a parameter grid, plot, report the optimum	`parseSweep`, `sweepPoint`, `sweepMeasurementTime`, `calculateSNRPlot`

Key idea

Steps 3–4 are the heart of the code. Everything that depends on ϕ is pulled out of the expensive part: the expensive part produces two *complex, ϕ-independent* correlator tables (C^{aa} and $C^{a^\dagger a}$ below), and ϕ only enters afterwards through cheap prefactors $\mathrm{e}^{-2\mathrm{i}\phi}$. That is why optimising ϕ costs essentially nothing.

Notation

Symbol	Meaning
N	cavity Fock-space truncation (module variable `N`)
$d = 4N$	Hilbert-space dimension; the tensor order is cavity ⊗ D ⊗ S
$K = d^2$	dimension of Liouville space (length of vec ρ)
$\kappa,\ \kappa_c$	total cavity decay rate, and the part that goes into the measured port
$j \in \{g, e\}$	initial state of S: ground (`'ground'`) or excited (`'excited'`)
n	number of time points (`tRes`); grid $t_i = i\,\Delta t$, $\Delta t = t_{\max}/(n-1)$
$X_\phi = \mathrm{e}^{-\mathrm{i}\phi}a + \mathrm{e}^{\mathrm{i}\phi}a^\dagger$	cavity quadrature measured by homodyne detection at phase ϕ
$\bar{a}_j(t) = \langle a(t)\rangle_j$	mean cavity amplitude for initial state j
A^{H}	conjugate transpose of a *matrix* (in contrast to $A^\dagger$ for an operator; they coincide numerically)

2 The quantities to be computed

2.1 Signal, noise, SNR

After removing the coherent drive from the input field, the measured, time-integrated quadrature is

$$M(\tau) = \int_0^\tau X_{\rm out}(t)\,dt, \qquad X_{\rm out} = X_{\rm in} + \sqrt{\kappa_c}\,X_\phi, \tag{1}$$

where $X_{\rm in}$ is vacuum noise on the measured port. Only the *measured* port rate κ_c appears here; internal losses and other ports enter only through the total κ inside the Lindbladian. (This is what makes κ_c/κ act like a detection efficiency η.)

Signal. The vacuum part has zero mean, so

$$\langle M(\tau)\rangle_j = \sqrt{\kappa_c}\int_0^\tau \langle X_\phi(t)\rangle_j\,dt, \qquad \langle X_\phi\rangle_j = 2\,\mathrm{Re}\Big[\mathrm{e}^{-\mathrm{i}\phi}\bar a_j(t)\Big], \tag{2}$$

and the distinguishing quantity is $\Delta M = \langle M\rangle_e - \langle M\rangle_g$. Writing the complex integral of the amplitude difference as

$$D(\tau) = \int_0^\tau \big[\bar a_e(t) - \bar a_g(t)\big]dt, \qquad \Rightarrow \qquad \Delta M(\tau) = 2\sqrt{\kappa_c}\,\mathrm{Re}\Big[\mathrm{e}^{-\mathrm{i}\phi}D(\tau)\Big]. \tag{3}$$

The only dynamical input is therefore $\bar a_j(t)$, which follows from the Lindblad equation alone.

Noise. Expanding the output correlation function with vacuum input (Gardiner–Collett), the variance for $t > t'$ becomes (Eq. (16) of the notes)

$$\sigma_j^2(\tau) = \tau + \kappa_c\int_0^\tau\int_0^\tau C_j(t,t')\,dt\,dt', \qquad C_j(t,t') = \Big\langle T{:}X_\phi(t)X_\phi(t'){:}\Big\rangle_j - \langle X_\phi(t)\rangle_j\langle X_\phi(t')\rangle_j. \tag{4}$$

The term τ is shot noise (vacuum). The second term is the excess noise of the system itself; it vanishes if the cavity is in a coherent state.

SNR. Eq. (18) of the notes,

$$\mathrm{SNR}(\tau) = \frac{|\Delta M(\tau)|}{\sigma_g(\tau) + \sigma_e(\tau)}, \qquad \tau_m:\ \mathrm{SNR}(\tau_m) = 1. \tag{5}$$

2.2 Separating out the phase ϕ

The symbol $T{:}\,{:}$ means normal ordering (all $a^\dagger$ left of all a) combined with time ordering (among the a's the later time stands left; among the $a^\dagger$'s the later time stands right). For $t > t'$, inserting $X_\phi = \mathrm{e}^{-\mathrm{i}\phi}a + \mathrm{e}^{\mathrm{i}\phi}a^\dagger$ gives four terms (Eq. (15) of the notes):

$$\Big\langle T{:}X_\phi(t)X_\phi(t'){:}\Big\rangle = \mathrm{e}^{-2\mathrm{i}\phi}\langle a(t)a(t')\rangle + \langle a^\dagger(t)a(t')\rangle + \langle a^\dagger(t')a(t)\rangle + \mathrm{e}^{2\mathrm{i}\phi}\langle a^\dagger(t')a^\dagger(t)\rangle. \tag{6}$$

The last two terms are complex conjugates of the first two, e.g. $\langle a^\dagger(t')a(t)\rangle = \langle a^\dagger(t)a(t')\rangle^*$, because $(a^\dagger(t)a(t'))^\dagger = a^\dagger(t')a(t)$. The product of means has exactly the same four-term structure with $\langle\cdot\cdot\rangle$ replaced by products of averages. Define the two **connected correlators**

$$\boxed{C_j^{aa}(t,t') = \langle a(t)a(t')\rangle_j - \bar a_j(t)\bar a_j(t'), \qquad C_j^{a^\dagger a}(t,t') = \langle a^\dagger(t)a(t')\rangle_j - \bar a_j^*(t)\,\bar a_j(t')} \qquad (t > t'). \tag{7}$$

Then

$$C_j(t,t') = 2\,\mathrm{Re}\big[\mathrm{e}^{-2\mathrm{i}\phi}\,C_j^{aa}(t,t')\big] + 2\,\mathrm{Re}\big[C_j^{a^\dagger a}(t,t')\big]. \tag{8}$$

Consequences. (i) C_j^{aa} and $C_j^{a^\dagger a}$ do not depend on ϕ; (ii) if the cavity is in a coherent state, both vanish (normally ordered moments factorise), so $\sigma_j^2 = \tau$; (iii) $\phi \to \phi + \pi$ flips the sign of ΔM but leaves C_j and σ_j unchanged, so scanning $\phi \in [0, \pi)$ covers all cases.

The remaining task is to compute $\bar{a}_j(t)$ and the two-time averages $\langle a(t)a(t')\rangle$, $\langle a^\dagger(t)a(t')\rangle$ for both initial states j.

3 Vectorisation: turning the Lindblad equation into a matrix

3.1 The master equation

With the operators of the code, the Lindblad equation is $\dot{\rho} = \mathcal{L}\rho$,

$$\mathcal{L}\rho = -\mathrm{i}[H, \rho] + \sum_{k=1}^{5} \Big(c_k \rho c_k^\dagger - \tfrac{1}{2}\{c_k^\dagger c_k, \rho\} \Big), \tag{9}$$

with collapse operators (`makeCollapseOps`)

$$c_1 = \sqrt{\kappa}\, a, \quad c_2 = \sqrt{\Gamma_1^D}\, \sigma_-^D, \quad c_3 = \sqrt{\Gamma_1^S}\, \sigma_-^S, \quad c_4 = \sqrt{\Gamma_\phi^D/2}\, \sigma_z^D, \quad c_5 = \sqrt{\Gamma_\phi^S/2}\, \sigma_z^S.$$

(Pure dephasing $\frac{\gamma}{2}\mathcal{D}[\sigma_z]$ makes the qubit coherence decay as $\mathrm{e}^{-\gamma t}$.) The Hamiltonian contains the drive $\mathrm{i}\sqrt{\kappa_c}\,\beta(a - a^\dagger)$, so the coherent input is already inside H and the input field is pure vacuum.

3.2 Column-stacking vectorisation

A $d \times d$ density matrix is flattened into a vector of length $K = d^2$ by stacking its columns,

$$\mathrm{vec}(\rho)_{\,r+d\,c} = \rho_{rc} \qquad (r = \text{row},\ c = \text{column}). \tag{10}$$

In NumPy this is `rho.reshape(-1, order='F')`. With this convention the product of three matrices becomes a Kronecker product,

$$\boxed{\mathrm{vec}(A\rho B) = (B^T \otimes A)\ \mathrm{vec}(\rho)} \tag{11}$$

Proof. $(A\rho B)_{rc} = \sum_{r',c'} A_{rr'}\rho_{r'c'}B_{c'c}$. The vector index is $r + dc$, with c the slow (outer) index, and $(B^T \otimes A)_{(c,r),(c',r')} = B^T_{cc'}A_{rr'} = B_{c'c}A_{rr'}$. □

Applying (11) term by term gives the Liouvillian as a $K \times K$ matrix:

$$L = -\mathrm{i}\big(\mathbb{1} \otimes H - H^T \otimes \mathbb{1}\big) + \sum_k \Big[c_k^* \otimes c_k - \tfrac{1}{2}\,\mathbb{1} \otimes c_k^\dagger c_k - \tfrac{1}{2}\,(c_k^\dagger c_k)^T \otimes \mathbb{1} \Big], \tag{12}$$

where $c^* = (c^\dagger)^T$ is the entrywise complex conjugate. The three types of term come from $H\rho$ ($B = \mathbb{1}$), ρH ($A = \mathbb{1}$) and $c\rho c^\dagger$ ($A = c$, $B = c^\dagger$). This is exactly `buildLiouvillian`:

```
L = -1j * (kron(I, Hs) - kron(Hs.T, I))
for c in c_ops:
    cdc = c.conj().T @ c
    L = L + kron(c.conj(), c) - 0.5*kron(I, cdc) - 0.5*kron(cdc.T, I)
```

Hence $\mathrm{vec}\,\rho(t) = \mathrm{e}^{Lt}\,\mathrm{vec}\,\rho(0)$. Because $\mathcal{L}$ generates a trace-preserving contraction, all eigenvalues satisfy $\mathrm{Re}\,\lambda \le 0$, and the eigenvalue 0 belongs to the steady state.

3.3 Expectation values are dot products

For any operator O,

$$\mathrm{Tr}[O\rho] = \sum_{i,j} O_{ij}\rho_{ji} = \sum_{i,j} O_{ij}\ \mathrm{vec}(\rho)_{\,j+di} = u_O^T\ \mathrm{vec}(\rho), \qquad u_O = \text{row-major flattening of } O. \tag{13}$$

In NumPy: `O.reshape(-1)` (default C order). Thus $\bar{a}_j(t) = u_a^T \,\mathrm{vec}\,\rho_j(t)$, and the population of the top Fock level or the trace are obtained the same way (`P_top`, identity), which gives the truncation and accuracy diagnostics for free. The dual statement is useful below: for matrices,

$$\mathrm{Tr}[A\,Y] = \mathrm{vec}(A^\dagger)^{\mathsf{H}}\ \mathrm{vec}(Y) = \big\langle\, \mathrm{vec}(A^\dagger)\,\big|\,\mathrm{vec}(Y)\big\rangle, \qquad \langle x|y\rangle := x^{\mathsf{H}}y. \tag{14}$$

(Check: $\mathrm{vec}(A^\dagger)_{\,r+dc} = (A^\dagger)_{rc} = A^*_{cr}$, so conjugating and summing against Y_{rc} gives $\sum A_{cr}Y_{rc} = \mathrm{Tr}[AY]$.)

4 Quantum regression on a time grid

4.1 The regression theorem

For two system operators A, B and a delay $s \geq 0$ (Eq. (17) of the notes),

$$\langle A(t'+s)\,B(t')\rangle = \mathrm{Tr}\Big[A\,\mathrm{e}^{\mathcal{L}s}\big(B\,\rho(t')\big)\Big]. \tag{15}$$

In words: propagate the *operator-multiplied* state $B\rho(t')$ for a time s with the same Lindbladian, then take the trace against A. (The theorem holds because the bath is Markovian and starts uncorrelated; the same $\mathcal{L}$ that gives one-time averages then gives two-time averages.) With $B = a$ and $A = a$ or $A = a^\dagger$ we obtain exactly the two averages needed in (7).

4.2 Discretisation

Use the uniform grid $t_i = i\Delta t$, $i = 0, \ldots, n-1$ (`tList = linspace(0, tMax, tRes)`). Let $t' = t_i$ be the *earlier* time and $s = t_k$ the delay. Define the two $n \times n$ tables

$$G^{aa}_{ik} = \mathrm{Tr}\big[a\,\mathrm{e}^{\mathcal{L}t_k}(a\,\rho(t_i))\big] = \langle a(t_i+t_k)\,a(t_i)\rangle, \qquad G^{a^\dagger a}_{ik} = \mathrm{Tr}\big[a^\dagger\,\mathrm{e}^{\mathcal{L}t_k}(a\,\rho(t_i))\big] = \langle a^\dagger(t_i+t_k)\,a(t_i)\rangle. \tag{16}$$

Because the grid is uniform, $t_i + t_k = t_{i+k}$, so the connected correlators (7) are

$$C^{aa}_{ik} = G^{aa}_{ik} - \bar{a}_{i+k}\,\bar{a}_i, \qquad C^{a^\dagger a}_{ik} = G^{a^\dagger a}_{ik} - \bar{a}^*_{i+k}\,\bar{a}_i, \qquad (i+k \leq n-1). \tag{17}$$

Only the triangle $i + k \leq n - 1$ lies on the grid; the rest is set to zero in the code (`valid` mask, index array `idx`). Both G tables are needed for *both* initial states j.

Key idea

The two backends below differ only in *how* G^{aa} and $G^{a^\dagger a}$ are computed. Everything before (`buildLiouvillian`) and after (connected correlators, integrals, SNR) is shared. Both backends return exactly the same arrays, which is verified numerically to $\sim 10^{-13}$.

5 Backend 1: diagonalisation (`method='eig'`)

5.1 Idea

Diagonalise the (non-Hermitian) matrix once, $L = V\,\mathrm{diag}(\lambda)\,V^{-1}$, with eigenvalues λ_m and right eigenvectors as columns of V. Every propagator is then a sum of exponentials:

$$\mathrm{vec}\,\rho(t) = V\,\mathrm{e}^{\Lambda t}\,c, \qquad c = V^{-1}\,\mathrm{vec}\,\rho(0), \qquad \Lambda = \mathrm{diag}(\lambda). \tag{18}$$

5.2 Read-out vectors

Combining (13) and (18),

$$\langle O\rangle(t) = u_O^T V\,\mathrm{e}^{\Lambda t} c = \sum_m w_m^O\,\mathrm{e}^{\lambda_m t}\,c_m, \qquad w^O = u_O^T V. \tag{19}$$

This is the method `LiouvillianSpectrum.readout` (computing w^O) and `expectation` (evaluating the sum on the grid).

5.3 The two-time tables

The state $a\rho(t_i)$ is first formed for all i at once: $\mathrm{vec}(\rho(t_i))$ are the columns of $R = V\,[\,c_m \mathrm{e}^{\lambda_m t_i}]$ (a $K \times n$ array, method `states`); then $\mathrm{vec}(a\rho(t_i)) = (\mathbb{1} \otimes a)\,\mathrm{vec}(\rho(t_i))$ is done by reshaping each column to a $d \times d$ matrix and multiplying by a from the left. Expand $a\rho(t_i)$ in eigenvectors,

$$Y_{mi} = \left[V^{-1}\,\mathrm{vec}(a\rho(t_i))\right]_m, \tag{20}$$

(using an LU factorisation of V, `lu_solve`, rather than the inverse), so that propagating for a time t_k just multiplies each component by $\mathrm{e}^{\lambda_m t_k}$:

$$G^A_{ik} = \sum_{m=1}^{K} Y_{mi}\,w^A_m\,\mathrm{e}^{\lambda_m t_k} \quad\Longleftrightarrow\quad G^A = \left(Y \circ w^A\right)^T T, \qquad T_{mk} = \mathrm{e}^{\lambda_m t_k}, \tag{21}$$

where $\circ$ multiplies row m of Y by w^A_m. This is a single $(n \times K)(K \times n)$ matrix product for each of $A = a$ and $A = a^\dagger$ (`twoTime`):

```
Y = lu_solve(luV, BR) # BR = vec(a rho(t_i)) as columns
T = exp(outer(lam, tList)) # T[m,k] = exp(lam_m t_k)
G = (Y * readout(A)[:, None]).T @ T # G[i,k] = <A(t_i+t_k) a(t_i)>
```

5.4 Cost, diagnostics, caveats

- **Cost.** One dense eigendecomposition: $\mathcal{O}(K^3) = \mathcal{O}(d^6) = \mathcal{O}\big((4N)^6\big)$ time and $\mathcal{O}(K^2)$ memory (for L, V and the LU copy). The two initial states share it. Afterwards the work is $\mathcal{O}(K^2 n + K n^2)$, i.e. cheap; the choice of `tRes` hardly matters.
- **Diagnostics.** $\max \mathrm{Re}\,\lambda_m$ must be 0 up to rounding, and $\mathrm{Tr}\,\rho(t) = 1$ must hold; `regressionData` warns otherwise.
- **Conditioning.** L is not normal, so V is not unitary and its condition number can be large. Near an exceptional point (two eigenvectors nearly parallel), V becomes ill-conditioned and the expansion loses digits. This is the main numerical weakness of this method and is what the trace/abscissa diagnostics detect.

6 Backend 2: adjoint propagation (`method='sparse'`, default)

6.1 Moving the propagator onto A

Combine (14) with $Y = \mathrm{e}^{\mathcal{L}s}(X)$:

$$\mathrm{Tr}\big[A\,\mathrm{e}^{\mathcal{L}s}(X)\big] = \big\langle \mathrm{vec}(A^\dagger)\,\big|\,\mathrm{e}^{Ls}\,\mathrm{vec}(X)\big\rangle = \big\langle \mathrm{e}^{L^{\mathsf{H}}s}\,\mathrm{vec}(A^\dagger)\,\big|\,\mathrm{vec}(X)\big\rangle. \tag{22}$$

The second equality is the definition of the matrix adjoint: $x^{\mathsf{H}}\mathrm{e}^{Ls}y = \big(\mathrm{e}^{L^{\mathsf{H}}s}x\big)^{\mathsf{H}}y$. Define, for each A, the vectors

$$w_k^A = \mathrm{e}^{L^{\mathsf{H}}t_k}\,\mathrm{vec}(A^\dagger) \in \mathbb{C}^K, \qquad k = 0,\dots,n-1. \tag{23}$$

Then, with $X = a\rho(t_i)$,

$$G_{ik}^A = \big(w_k^A\big)^{\mathsf{H}}\,\mathrm{vec}\big(a\rho(t_i)\big) \quad\Longleftrightarrow\quad G^A = \Big(\overline{W^A}\,\mathrm{BR}\Big)^T, \tag{24}$$

where row k of W^A is $(w_k^A)^T$ and column i of BR is $\mathrm{vec}(a\rho(t_i))$.

Physical meaning. L^{H} is the matrix of the *Heisenberg-picture* generator

$$\mathcal{L}^\dagger(X) = \mathrm{i}[H,X] + \sum_k \Big(c_k^\dagger X c_k - \tfrac{1}{2}\{c_k^\dagger c_k, X\}\Big),$$

so $w_k^A = \mathrm{vec}\,\big(A(t_k)^\dagger\big)$ with the Heisenberg-evolved operator $A(t) = \mathrm{e}^{\mathcal{L}^\dagger t}A$ (the generator preserves Hermitian conjugation, $\mathcal{L}^\dagger(X)^\dagger = \mathcal{L}^\dagger(X^\dagger)$), and (22) is just the Heisenberg–Schrödinger duality $\mathrm{Tr}[A\,\mathrm{e}^{\mathcal{L}s}X] = \mathrm{Tr}[A(s)\,X]$.

Key idea

Why this is cheaper. In QuTiP's `correlation_2op_2t` every one of the n states $a\rho(t_i)$ is propagated separately for up to n steps by an ODE solver, and in Backend 1 each of them has to be expanded in the eigenbasis (a solve with the $K \times K$ matrix V against n vectors). In (23) the operator A is propagated instead, and it is the *same* for all i. So only **one vector per** A ($a^\dagger$ and a, two in total) must be propagated with L^{H}, plus **one vector per initial state** ($\rho_g(0)$, $\rho_e(0)$) with L. The n-dependence is then confined to one cheap matrix product.

6.2 Propagating a vector on a whole time grid

For a sparse matrix M, a vector v and the equispaced grid t_k, SciPy's `expm_multiply` returns $\mathrm{e}^{Mt_k}v$ for all k without ever forming e^{Mt}:

```
rho_t = expm_multiply(L, B, start=0, stop=T, num=n, endpoint=True) # (n, K, 2): e^{L
    t_k} vec(rho_0^{(j)})
W = expm_multiply(Lh, adj, start=0, stop=T, num=n, endpoint=True) # (n, K, 2): e^{L^H
    t_k} [vec(a^dag), vec(a)]
```

(the last axis stacks the two initial states, respectively the two operators $A = a$, $A = a^\dagger$; note $\mathrm{vec}(A^\dagger) = \mathrm{vec}(a^\dagger)$ for $A = a$, which is why the stacked columns are `[vec(ad), vec(a)]`). Internally `expm_multiply` (Al-Mohy–Higham) splits the interval into steps small enough that a short truncated Taylor series of $\mathrm{e}^{M\Delta}$ applied to the vector is accurate to machine precision, using only sparse matrix–vector products.

6.3 Assembling and cost

With $R_i = \mathrm{vec}\,\rho_j(t_i)$ from the first call: $\bar{a}_j(t_i) = u_a^T R_i$, the diagnostics use the same dot products, $\mathrm{vec}(a\rho(t_i))$ is obtained by reshaping to $d \times d$ and multiplying by a, and (24) gives

```
Gaa = (W[:, :, 0].conj() @ BR).T # A = a
Gada = (W[:, :, 1].conj() @ BR).T # A = a^dag
```

- **Time.** Each `expm_multiply` call needs a number of sparse matrix–vector products roughly proportional to $\|L\|_1\, t_{\max}$ (the 1-norm grows with detunings, couplings and with N through κN and $g\sqrt{N}$), each costing $\mathcal{O}(\mathrm{nnz}(L))$ with $\mathrm{nnz}(L) \propto K$ (see Sec. 10). The final product costs $\mathcal{O}(Kn^2)$.
- **Memory.** $\mathcal{O}(nK)$ for R and W; no $K \times K$ object is ever created.
- **Accuracy.** The truncation tolerance of `expm_multiply` is machine precision, and no ill-conditioned eigenvector matrix is involved, so this backend has no analogue of the exceptional-point problem.

6.4 Measured comparison

All numbers are for the default parameters on one core of the test machine; the diagonalisation time barely depends on `tRes`, the sparse time grows mildly with it.

N	$K = (4N)^2$	`method='eig'`	`method='sparse'`
10	1600	10.3 s	2.0 s
12	2304	21 s	3.3 s
16	4096	125 s	—
20	6400	(extrapolated: minutes, ∼3 GB)	9.8 s

(`eig`: `tRes`=400 for $N = 12, 16$, 200 for $N = 10$; `sparse`: `tRes`=200.) Both methods gave the same τ_m to all printed digits.

7 Integrating the correlators: the double integral

The excess noise needs $\iint_{[0,\tau]^2} C_j\, dt\, dt'$ for every τ on the grid. Since $C_j(t, t')$ is symmetric under $t \leftrightarrow t'$ (the output quadratures commute), the square is twice the triangle $t' < t$. Because (8) is linear in C_j^{aa} and $C_j^{a^\dagger a}$, the code integrates the two *complex* tables separately, once, and combines them with ϕ afterwards.

7.1 Definition

For a complex table C with entries $C_{ik} = C(t' = t_i,\, t = t_i + t_k)$ define

$$I(\tau) = \iint_{[0,\tau]^2} C = 2 \int_0^\tau dt \int_0^t dt'\, C(t, t'). \tag{25}$$

7.2 Algorithm (`doubleIntegralCumulative`)

1. **Inner integral.** For fixed upper time t_m, the points $(t', t) = (t_i, t_m)$, $i = 0..m$, are the entries $C_{i,\,m-i}$ – an *anti-diagonal* of the table. By the trapezoid rule

$$h_m = \int_0^{t_m} C(t_m, t')\, dt' \approx \Delta t \Big[\sum_{i=0}^{m} C_{i,m-i} - \tfrac{1}{2}\big(C_{0,m} + C_{m,0}\big) \Big]. \tag{26}$$

   In NumPy the anti-diagonals become ordinary diagonals after `C[:, ::-1]`, so `np.diagonal(A, offset=n-1-m)` extracts them.

2. **Outer integral.** The cumulative trapezoid of h,

$$F_m = \sum_{l=1}^{m} \tfrac{1}{2}\Delta t\,(h_l + h_{l-1}), \qquad F_0 = 0, \tag{27}$$

   is the triangle integral up to t_m.

3. **Symmetrise.** $I_m = 2F_m$.

The cost is $\mathcal{O}(n^2)$. I replaced `np.trapz` by this explicit rule so that complex arrays work and the code does not depend on the NumPy version.

Applying it to the two tables gives, for each initial state j, the complex arrays $I_j^{aa}(\tau_m)$ and $I_j^{a^\dagger a}(\tau_m)$, and (4) with (8) becomes

$$\boxed{\sigma_j^2(\tau) = \tau + \kappa_c \Big\{ 2\ \mathrm{Re}\left[\mathrm{e}^{-2\mathrm{i}\phi} I_j^{aa}(\tau)\right] + 2\ \mathrm{Re}\left[I_j^{a^\dagger a}(\tau)\right] \Big\}} \tag{28}$$

(`max(var,0)` is applied against rounding, since a variance cannot be negative).

Caveat

The trapezoid rule needs $\Delta t = t_{\max}/(n-1)$ to resolve the fastest oscillation in the correlation functions (roughly set by the qubit and cavity detunings in the rotating frame). With too few points the integral of an oscillating tail is wrong. Because the expensive step no longer scales with `tRes`, it is cheap to choose $n \gtrsim 200$.

8 Assembling the SNR, the phase, and the measurement time

8.1 Signal (`snrFromData`)

The complex amplitude integral $D(\tau)$ of (3) is a cumulative trapezoid of $\bar{a}_e - \bar{a}_g$ on the grid. Then for any phase

$$\Delta M(\tau_m; \phi) = 2\sqrt{\kappa_c}\ \mathrm{Re}\left[\mathrm{e}^{-\mathrm{i}\phi} D(\tau_m)\right]. \tag{29}$$

8.2 Noise and SNR for many phases at once

With $I_j^{aa}, I_j^{a^\dagger a}$ from the previous section, (28) gives $\sigma_j(\tau;\phi)$ and hence $\mathrm{SNR}(\tau;\phi) = |\Delta M|/(\sigma_g + \sigma_e)$. The code evaluates this for an array of phases simultaneously by broadcasting (arrays of shape $n_\phi \times n$); each phase costs $\mathcal{O}(n)$, which is negligible. At $\tau = 0$ the SNR is set to 0 (it is 0/0).

8.3 Choosing ϕ (`phi='auto'`)

Because of $\phi \to \phi + \pi$ symmetry, $n_\phi =$`phiRes` phases are scanned on $[0, \pi)$. For each phase the measurement time $\tau_m(\phi)$ is computed; the phase with the smallest τ_m is chosen. If no phase reaches SNR $= 1$ within $t_{\max}$, the phase with the highest SNR peak is chosen instead. A numeric `phi` skips the scan. The resolution of the scan is π/n_ϕ (default $\approx 2.8°$), which costs a negligible relative SNR error because the SNR is smooth in ϕ and stationary at the optimum.

8.4 Measurement time (`measurementTime`)

In the QND-like regime $\Delta M \propto \tau$ and $\sigma^2 \propto \tau$, so SNR^2 grows *linearly*. The first grid index k with $\mathrm{SNR} \geq 1$ is found, and SNR^2 is interpolated linearly between t_{k-1} and t_k:

$$\tau_m = t_{k-1} + \frac{1 - s_{k-1}^2}{s_k^2 - s_{k-1}^2}\,\Delta t, \qquad s_k = \mathrm{SNR}(t_k). \tag{30}$$

If SNR never reaches 1 before $t_{\max}$, $\tau_m = \infty$ (shown as blank or as a $\times$ on the upper edge in the plots). The code also reports $\max_\tau \mathrm{SNR}$, its location, and `GammaM`: the least-squares slope of SNR^2 over the last 30 % of the grid, which equals the measurement rate $\Gamma_m = d\,\mathrm{SNR}^2/d\tau$ in the QND-like regime (it tends to zero when the signal saturates).

8.5 Expected limits (consistency checks)

- **Coherent cavity** (e.g. $g = 0$): $C^{aa} = C^{a^\dagger a} = 0$, $\sigma_j^2 = \tau$.
- **Dispersive limit, no dephasing:** $C \approx 0$, $\sigma_j^2 \approx \tau$, and $\Gamma_m = \kappa_c\,|\bar{a}_e - \bar{a}_g|^2_{\mathrm{steady}}$, i.e. the toy-model formula (24) of the notes evaluated with the *true* steady-state amplitudes.

9 Sweeps

A sweep evaluates the whole chain above on a grid of parameter values and collects τ_m, $\max \mathrm{SNR}$, the optimal ϕ and `GammaM` at each point.

- `parseSweep`: a sweep is `[name, min, max, number]` or `[name, min, max, number, 'log']`. `'omegad'` and `'omegaq'` are aliases for `omega_d`, `omega_q` (table `SWEEP_ALIASES`); any other key of `params` (`beta`, `g`, `gd`, `kappac`, ...) can be swept by name. The values are `linspace` or, for `'log'`, `geomspace`.
- `sweepPoint`: one grid point; returns `None` if the linear algebra fails. It is a top-level function so that it can be pickled and sent to worker processes.
- `sweepMeasurementTime`: builds the list of jobs (a $1 \times n_1$ or $n_2 \times n_1$ grid), runs them serially or in a `ProcessPoolExecutor`, assembles arrays of shape (n_2, n_1), converts "never reached" to `nan`, and finds the best point (`nanargmin` of τ_m). It also warns if the top Fock level population exceeded 10^{-4} at any point.
- `initWorker`: each worker restricts BLAS/OpenMP to one thread (via `threadpoolctl` if installed), because otherwise n_{jobs} workers each start one thread per core and compete.
- `sweepAxis`: converts `omegad` to $\Delta_r = \omega_r - \omega_d$ and `omegaq` to $\omega_r - \omega_q$ for plotting.
- `calculateSNRPlot`: the user-facing function. With `sweep=['none']` it plots $\mathrm{SNR}(\tau)$ and marks τ_m; for a 1D sweep it shows a colour map of $\mathrm{SNR}(\tau)$ against the parameter with the

SNR = 1 contour, and τ_m against the parameter (points where SNR = 1 is not reached are drawn as × at $t_{\max}$); for a 2D sweep it shows maps of τ_m (log colour scale, minimum starred) and of max SNR. It writes a PDF, a parameter file and an `.npz` with all arrays to the folder `calculateSNRPlot`, and prints the optimum.

Cost of a sweep is (number of points)×(cost of one point); there is no shared work between points because L changes with every parameter. With `method='sparse'` one point costs seconds even at $N = 20$; with `'eig'` it is dominated by the $\mathcal{O}(K^3)$ diagonalisation.

10 Dense versus sparse matrices, and the other options

10.1 What the two representations are

A **dense** matrix stores every one of its K^2 entries, including the zeros. A **sparse** matrix stores only the nonzero entries and where they are. In the CSR (compressed sparse row) format used here, three arrays are kept: `data` (the nonzero values), `indices` (their column numbers) and `indptr` (where each row starts). Memory is $\approx \text{nnz} \times (16 + 4)$ bytes for complex values, instead of $16K^2$.

10.2 Why the Liouvillian is very sparse

Eq. (12) is a sum of Kronecker products of $d \times d$ operators, and each operator (a, $a^\dagger a$, σ_z, the qubit operators, H) couples every basis state to only a handful of others. A Kronecker product of such matrices again has only a handful of entries per row. For the default parameters (measured, using the code's own matrix):

N	K	nonzeros	per row	dense memory	CSR memory
12	2304	19 790	8.6	85 MB	0.4 MB
20	6400	56 654	8.9	655 MB	1.2 MB

The number of nonzeros *per row stays about 9 independent of* N, so $\text{nnz} \propto K$ while a dense matrix grows as K^2.

10.3 What each representation is good for

	Dense (`buildLiouvillian(..., dense=True)`)	**Sparse** (default, CSR)
Memory	$16K^2$ bytes	$\sim 20\,\text{nnz}$ bytes
Matrix–vector product	$\mathcal{O}(K^2)$	$\mathcal{O}(\text{nnz}) \sim \mathcal{O}(K)$
Full eigendecomposition	**possible** (LAPACK `la.eig`)	**not available**: sparse solvers (`eigs`) return only a few eigenpairs, and eigenvectors of a sparse matrix are dense anyway
Used by	`method='eig'`	`method='sparse'` (`expm_multiply`)

So the choice is dictated by the algorithm: diagonalisation *needs* the dense matrix because all K eigenpairs are required; adjoint propagation *only needs matrix–vector products*, for which sparse storage is faster by about $K/(\text{nonzeros per row})$, i.e. $\approx 700\times$ at $N = 20$. Passing a dense matrix

to `expm_multiply` would still work but would waste that factor and the memory. The parameter `dense` of `buildLiouvillian` (default `False`) only switches the return type; `LiouvillianSpectrum` requests `dense=True` internally, the sparse backend uses the default.

10.4 Other sparse formats (and what the code does)

Format	Stores / strength	Role here
COO	list of (row, col, value); cheap to build and sum	what `kron` returns internally; not good for arithmetic
CSR	row-compressed; fast row slicing and matrix–vector products	**used**: `L.tocsr()` once after construction
CSC	column-compressed; fast column slicing, the transpose of CSR	what `.conj().T` of a CSR matrix produces; converted back with `.tocsr()` for L^{H}
LIL / DOK	easy incremental editing	not used (only useful while assembling by hand)
DIA / BSR	diagonals / dense blocks; good for banded or block matrices	not used; L has no simple band structure

10.5 Other ways to get the same two-time tables

Approach	Cost	Comment
QuTiP `correlation_2op_2t` (original)	n separate master-equation solves, per correlator and per initial state	general, but slow; kept as `CalculateC_qutip` for validation
Diagonalisation (`'eig'`)	$\mathcal{O}(K^3)$ once	exact sums of exponentials; shares work between both initial states; limited by N and by conditioning of V
Adjoint `expm_multiply` (`'sparse'`)	$\sim \mathrm{nnz} \cdot \|L\| \, t_{\max}$	**default**; fast, low memory, no ill-conditioning
Dense matrix exponential (`scipy.linalg.expm`) of $L\,\Delta t$ and repeated multiplication	$\mathcal{O}(K^3)$ to build, then $\mathcal{O}(K^2)$ per step per vector	simple but needs the dense matrix; not implemented
Matrix-free (never build L; apply $\mathcal{L}$ to a $d \times d$ matrix with sparse H and c_k)	$\mathcal{O}(d \cdot \mathrm{nnz}(H))$ per application	lowest memory; would allow still larger N; not implemented

11 Reference of all options

11.1 Module-level switches

Name	Effect
`N`	Fock-space truncation (default 12, was 20). Set it by the photon number: N must be well above the largest occupied Fock level. The code warns if the top-level population exceeds 10^{-4}. A too small N is not merely inaccurate: it produces spurious excess noise (for $g = 0$, where C must vanish, I measured $\max\|C^{aa}\| \approx 2 \times 10^{-2}$, 1×10^{-3}, 3×10^{-5} for $N = 6, 8, 10$).
`USE_LATEX`	`True` uses LaTeX for plot text (needs a LaTeX installation); set `False` otherwise.

11.2 Options of `calculateSNR`, `regressionData`, `calculateSNRPlot`

Option	Meaning and guidance
`params`	Parameter dictionary; derived quantities (mixing coefficients C, S) are added by `withDerived`.
`method`	`'sparse'` (default): adjoint propagation, fast, scales to $N = 20$. `'eig'`: diagonalisation, same results, cost $\sim (4N)^6$; use as an independent cross-check or when many different τ grids are needed on one L.
`phi`	A number: fixed homodyne phase. `'auto'`: scan and pick the phase with the shortest τ_m (recommended, in particular for sweeps, because the optimal phase moves with the parameters).
`phiRes`	Number of phases in the scan on $[0, \pi)$ (default 64).
`tMax`	Longest integration time. Choose it well above the expected τ_m; points that do not reach SNR $= 1$ before $t_{\max}$ are shown as "not reached". Larger `tMax` costs more matrix–vector products (linear).
`tRes`	Number of time points; $\Delta t = t_{\max}/(\texttt{tRes} - 1)$ must resolve the oscillations in the correlators (trapezoid rule). Memory for the correlator tables is $2 \times 2 \times n^2 \times 16$ bytes (≈ 10 MB at $n = 400$, ≈ 250 MB at $n = 2000$).
`returnInfo`	`calculateSNR` also returns a dictionary with `phi`, `tauM`, `snrMax`, `tauAtMax`, `GammaM`, `topFockPop`, `traceError`, `spectralAbscissa` (the last is only defined for `'eig'`).
`warn`	Emit warnings for a too small Fock space ($> 10^{-4}$), trace error ($> 10^{-6}$) or $\max \mathrm{Re}\,\lambda > 10^{-8}$ (`'eig'`). Switched off inside sweeps, which warn once at the end.
`sweep`	`['none']`, or `[name, min, max, number]` (optionally `, 'log'`).
`sweep2`	Optional second sweep (same format) for a 2D map; must be a different parameter.
`nJobs`	Number of worker processes for sweeps (default 1). Points are independent, so the speed-up is close to linear in the number of cores; memory is multiplied by `nJobs`. Needs `if __name__ == '__main__':` on Windows/macOS. Install `threadpoolctl` (or set `OMP_NUM_THREADS=1`) to avoid thread oversubscription.

11.3 Other functions' options

Option	Meaning
`buildLiouvillian(..., dense)`	Return a dense `ndarray` (`True`) or sparse CSR matrix (`False`, default); see Sec. 10.
`measurementTime(..., threshold)`	SNR level that defines τ_m (default 1).
`initialDensityMatrix(rho0)`	`'ground'` or `'excited'`: S in $\|j\rangle$, D in its ground state, empty cavity.
`compareWithQutip(params, phi, tMax, tRes, method)`	Runs the QuTiP reference and the new code on a small grid and prints the maximal deviations of the correlator tables and of the SNR curve; keep `tRes` small because the QuTiP part is slow.

12 Accuracy, limits and validation

12.1 Sources of error

1. **Fock truncation** (N): see above; check `topFockPop`.
2. **Time discretisation** (`tRes`): the trapezoid error is $\mathcal{O}(\Delta t^2)$; halve Δt and check that τ_m is stable.
3. **Conditioning** (`'eig'` only): near exceptional points V is ill-conditioned; the trace error and spectral abscissa expose this.
4. **Gaussian assumption:** the SNR presumes roughly Gaussian histograms of M. Near resonance this may fail; the SNR is then still a well-defined figure of merit for a linear detector, but not necessarily optimal (this is a property of the formula, not of the numerics).
5. **Window:** if $\tau_m > t_{\max}$ the point is reported as not reached; this is not an error but a request for larger `tMax`.

12.2 What was checked

- **Against an independent reference.** The correlator tables and SNR curves of both backends agree with a reference that integrates the master equation as an ODE (re-implementing the semantics of `mesolve` and `correlation_2op_2t`) to about 10^{-12}, for the default parameters and for a detuned, strongly flip-flop-coupled, relaxing parameter set. The two backends agree with each other to about 10^{-13}. **QuTiP itself was not available in the test environment**, so please run `compareWithQutip(params)` once on your machine.
- **Coherent state.** For $g = 0$ the correlators vanish up to truncation error, which converges to zero as N grows (numbers above).
- **Dispersive limit** (recipe step 6 of the notes): with dephasing switched off, the late-time slope of SNR^2 equals $\kappa_c|\bar{a}_e - \bar{a}_g|^2$ to 0.3 % (ratios 0.9986 for $g = 0.25$, 0.9967 for $g = 0.5$), and $\sigma^2/\tau = 1.000$ to 0.4 %. The lowest-order formula (22)–(24) of the notes, with $\chi = g^2/(\omega_D + 2J_L s - \omega_r)$, is accurate to 3 % at $g/\Delta = 0.05$ and 14 % at 0.1, as expected from higher-order dispersive corrections.
- **With dephasing on** (D: $\gamma = 0.02$, $g/\Delta = 0.1$) the SNR^2 slope was about 0.62 of the lowest-order prediction. This is explained quantitatively by (a) the signal difference being 0.85 of the lowest-order value and (b) real excess noise ($\sigma^2/\tau \approx 1.07$–$1.22$ at the chosen phase) from rare dressed-state flips of D caused by its dephasing; i.e. it is the physical $C_j \neq 0$ effect of Eq. (16), not a numerical artefact.

13 Map from mathematics to code

Function / class	Computes	Equations
`initialDensityMatrix`	$\rho_j(0)$	—
`buildLiouvillian`	L (sparse or dense)	(11), (12)
`LiouvillianSpectrum`	V, λ, c, w^O, $\langle O\rangle(t)$, G^A by sums of exponentials	(18)–(21)
`_regressionEig`	$\bar{a}_j$, G^{aa}, $G^{a^\dagger a}$ and diagnostics, via diagonalisation	(16), (21)
`_regressionSparse`	same, via adjoint propagation	(22)–(24)
`regressionData`	connected correlators, double integrals, checks	(17)
`doubleIntegralCumulative`	$I(\tau)$ from a table	Sec. 7
`snrFromData`	ΔM, σ_j, SNR, optimal ϕ, `GammaM`	(3), (28)
`measurementTime`	τ_m	Sec. 8
`calculateSNR`	wrapper (compatible with the old call)	—
`parseSweep`, `sweepAxis`, `sweepPoint`, `initWorker`, `sweepMeasurementTime`	parameter grids, parallel evaluation, best point	Sec. 9
`calculateSNRPlot`	figures, files, printed optimum	—
`CalculateC_qutip`, `calculateSNR_qutip`, `compareWithQutip`	original QuTiP implementation and the comparison	validation only